What's new in the Jakarta Turbine 2.3 Web Application Framework?

Henning Schmiedehausen
henning@apache.org
ApacheCon 2003, Las Vegas



The Apache Jakarta Project
http://jakarta.apache.org/

Ultra short Exec Summary

- Turbine is a single-point-of-entry servlet
- Turbine uses Templates as its view (not JSP, though JSPs are possible)
- Turbine supports many of the three/four-letter-acronyms and buzzwords needed to use it in a corporate environment

  (However, we don‘t support all of them (yet))



What is Turbine?

- Turbine is a servlet based framework for web applications
- It is based on the “Model 2+1” pattern
- “A toolbox for web application developers”
- Has a singleton based Service framework
- Contains an event driven Action model
- Uses Velocity for a template based View



A short history of Turbine

- Turbine traces back to the retired java.apache.org project
- Long, troublesome history caused by a “We will release when it‘s ready” attitude

  (Which is, as we all know, never)
- While being around for a long time, surpassed popularity-wise by other web frameworks (Struts, Cocoon)



Turbine sightings

- Oct 1999: Turbine 1.0Beta1 (derived from working-dogs.com Dash project)
- Project was pulled back into "Alpha, not released" state after that
- Jun 4th, 2001: Turbine 2.1 release
- Dec 8th 2002: Turbine 2.2 release
- Sep 4th 2003: Turbine 2.3 release

(We try to break the "one release per year cycle")

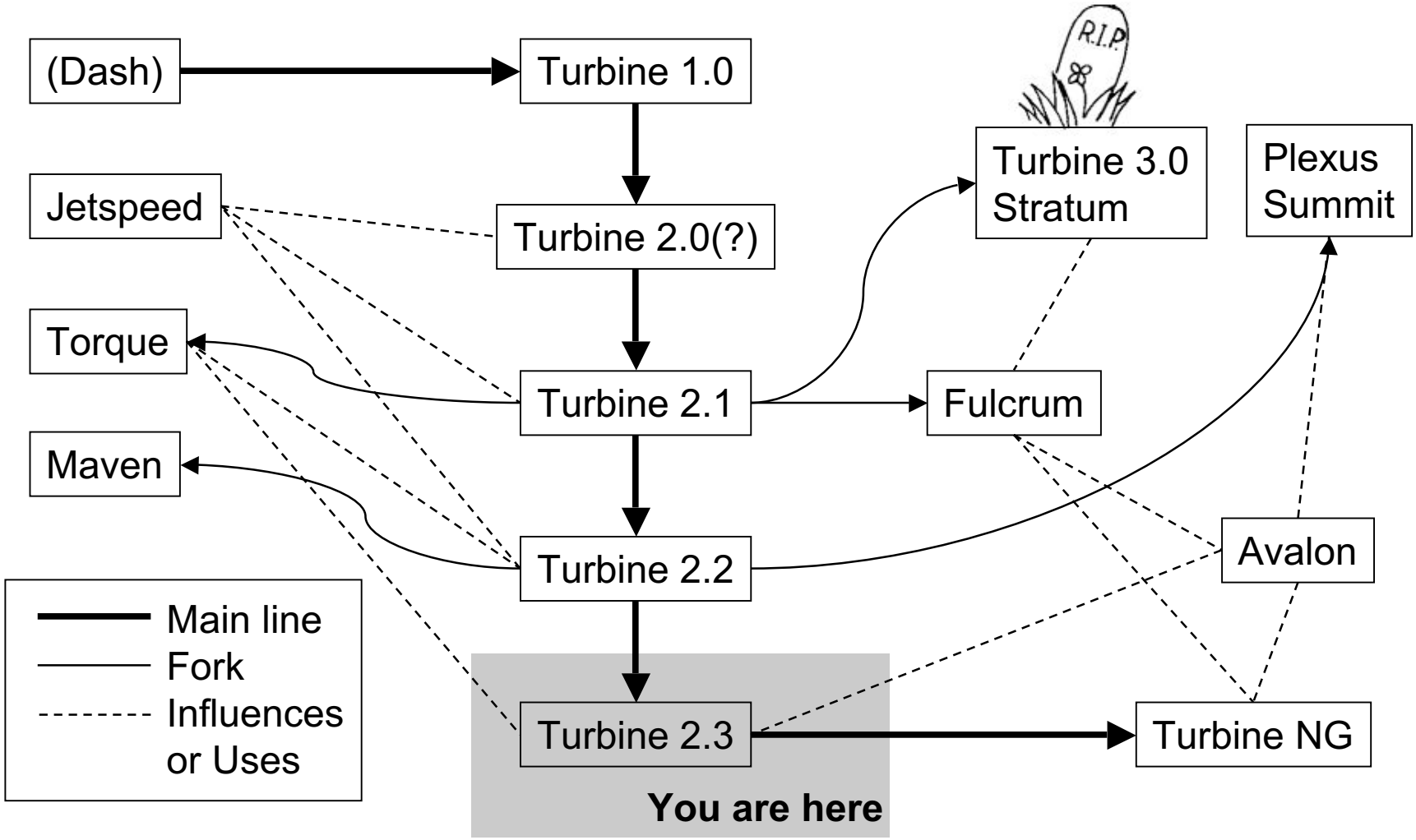


Turbine trouble

- Turbine was (ab)used as “playground”
- Lots of developers, almost no docs
- Repeated goal changes led to FUD
- Forking off (T2 / T3) didn’t help either
- Just CVS access and hard to rebuild
  (changed in the 2.3 development cycle)
- Unwieldy TDK, which isn’t even used by some of the developers



Turbine forks



Good news

- Significant change in development after the 2.2 release
- Some of the seasoned developers left, new developers arrived
- The focus is now back on being a web application framework
- We even have a documentation project
  (Thanks to Peter Courcoux and all the contributors)



Status

- The Turbine 3 branch and Stratum are considered dead by the core developers (Don’t use for new projects)
- Maven became its own top-level ASF project
- Some of the more radical ideas went off-ASF in the Plexus/Summit project



Turbine 2.3 Features

- New build system using Maven
- easier application integration (logging, configuration)
- deprecating aged or moved code
- Removal of WebMacro, FreeMarker and Castor support
- bug fixes (of course)



Turbine 2.3 Features (2)

- Crypto support
- Session Service
- Avalon based Component Service
- Template Service reworked and cleaned
- New URI management scheme
- Intake reworked and improved
- Security Service can use arbitrary Torque Peers or LDAP directories



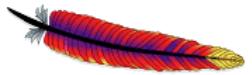
Building Turbine

- Both Source and Binary releases available
- Recompiling used to be cumbersome
- Building with Maven is the only official way
  (There is some preliminary support for building the source in the Eclipse IDE)
- Maven has some quirks but promises to be a very versatile tool in the future
- Ant users are welcome to donate code



Building Turbine with Maven

- All official releases starting with 2.3 can be built with the ibiblio.org jar repository
- Some jars are not yet available from there due to license reasons, they must be downloaded separately
  (Javamail, JAF, JNDI, JDBC)
- Supported JDK is 1.3 or better



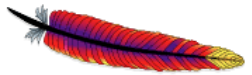
Logging Support

- T2.2 contained a Logging Service for application logging
- Much of the core code didn't contain any logging or used System.out.print for debug
- Logging had special cases for init, core and application logging
- Debugging with System.out.print was easier than using the logging service



Logging Support (2)

- T2.3 now uses commons-logging in an uniform manner
- log4j is used as logging implementation (which currently is hard coded but could easily made configurable)
- Turbine now provides an initialized and flexible logging system to the application
- The old logging service and the Log class are deprecated or have been removed



Logging Support (3)

- Logging is hierarchical, with inheritance, based on class and package names
- Default configuration file provides log files for the core, Velocity, Torque, Avalon based components and the Scheduler
- Extensive documentation for log4j is on http://jakarta.apache.org/log4j/
  (The log4j talk is right after the coffee break) ;-)



Turbine Logging

Use a class member for logging, if you

- use Logging in a static context
- don’t want to inherit from the class
- always want to have the same logging category

```
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;

public class Foo {
[...]
  private static final Log log = LogFactory.getLog(Foo.class);
[...]
}
```




Turbine Logging (2)

Use an instance member for logging if you

- want to have to change the logging category automatically if the class has been extended

```
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;

public class Foo {
[...]
  protected Log log = LogFactory.getLog(this.getClass());
[...]
}
```




Log4j Configuration for Turbine

The log4j configuration must contain a root logger and may contain one or more category based loggers

```
log4j.rootLogger = INFO, turbine
[...]
log4j.category.com.mycompany.app = DEBUG,app
log4j.additivity.com.mycompany.app = false
[...]
log4j.appender.app          = org.apache.log4j.FileAppender
log4j.appender.app.file     = ${applicationRoot}/logs/avalon.log
log4j.appender.app.layout = org.apache.log4j.PatternLayout
log4j.appender.app.layout.conversionPattern = %d [%t] %-5p %c - %m%n
log4j.appender.app.append = false
```




Configuration and Properties

- T2.2 contained a Resources Service for Core and Application configuration
- Using a Service had some bootstrapping issues with configuration
- Much of the Resource Service code (which came from jserv) became jakarta-commons-sandbox/configuration



Configuration and Properties (2)

- T2.3 now uses commons-configuration
- The Resources Service is deprecated and will be removed in the 2.4 cycle
- Configuration is loaded by the core at startup time and can be queried by the services and the application
- Current default is a properties based configuration



Configuration and Properties (3)

- commons-configuration has been greatly extended since its inception
- Configuration can e.g. be loaded from an XML file, from JNDI or from a combination of these
- There is some discussion about making the Turbine configuration Avalon compatible



Application Configuration

Turbine provides a static method for accessing the global configuration

```
import org.apache.commons.configuration.Configuration;
import org.apache.turbine.Turbine;

public class Foo {
[...]
  Configuration conf = Turbine.getConfiguration();
  boolean myFlag = conf.getBoolean("app.myflag", false);
  String myName = conf.getString("app.myname", "");
[...]
}
```




Service Configuration

Each Service can fetch its configuration subset (prefixed with services.<Service Name>) from the TurbineBaseService

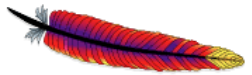
```
public class MyServiceImplementation
  extends TurbineBaseService implements MyService {

public void init() throws InitializationException {
    Configuration conf = getConfiguration();
    boolean serviceFlag = conf.getBoolean("serviceFlag", false);
    String serviceData = conf.getString("serviceData", "");
  }
}
```




Intake Service

- T2.3 contains nearly a complete rework of the form validation framework
- Intake can now load multiple XML definition files with prefixes
- reworked Validators return better and consistent error messages
- Default values and empty field behaviour have been improved



Crypto Service

- T2.2 had cryptography only in the Security Service for password encryption
- T2.2 could only work with MessageDigest ciphers (e.g. no Unix crypt), which made integration difficult (legacy databases, Apache httpd authentication)



Crypto Service (2)

- T2.3 offers an implementation which allows “plugging” of Crypto Algorithms
- Target is the encryption of passwords for databases and applications
- Turbine includes support for
  - all ciphers from java.security.MessageDigest (SHA, MD5,…)
  - Unix crypt



Crypto Service Usage

```
import org.apache.turbine.services.crypto.CryptoAlgorithm;
import org.apache.turbine.services.crypto.CryptoService;
import org.apache.turbine.services.crypto.TurbineCrypto;

public class PwManager {

  public boolean encryptPw(String password) throws Exception {
    CryptoService cs = TurbineCrypto.getService();
    CryptoAlgorithm md5Algo = cs.getCryptoAlgorithm("MD5");
    return md5Algo.encrypt(password);
  }
}
```




Session Service

- New in Turbine 2.3
- Allows an application to access information about all currently active sessions
- Makes it possible to list current active users or to avoid double logins
- Requires Servlet API 2.3 or better



Session Service Listener

The Session Service must register with the container by adding a listener to the application deployment descriptor

web.xml

```
[...]
<listener>
  <listener-class>
    org.apache.turbine.services.session.SessionListener
  </listener-class>
</listener>
[...]
```




Session Service Tool

Session Service provides a pull tool which can be used to list information

TurbineResources.properties

```
tool.session.sessionmgt = \
        org.apache.turbine.services.session.SessionTool
```

UserList.vm

```
<p>Users logged in:</p>
#foreach ($logUser in $!sessionmgt.ActiveUsers)
  $!logUser.Name ($!logUser.FirstName $!logUser.LastName)
#end
```




Avalon Component Service

- Prior to 2.3, Turbine could load only Components that use the Stratum lifecycle interfaces
- Unfortunately Stratum was never officially released (“We release when it’s ready”)
- About the only component ever loaded by Turbine has been Torque



Turbine and Avalon

- Avalon has a well thought out component model, that is widely accepted (James, Cocoon)
- It makes sense to access Avalon Components from Turbine applications
- Unfortunately, the current state of the Avalon project makes it hard to determine how to go on with it



Using Avalon Components

- Some of the Avalon concepts (IoC, Facade classes, Singleton Services) differ from Turbine
- T2.3 uses ECM as container to load and execute Avalon components
- Can be used to load Torque, removing the need for the old, Stratum based Component Service



Template Service

- Do not confuse with a templating Engine
- TemplateService is engine-independent, engines register with it
- However, its main usage is with Velocity Templates
- Prior to 2.3, the whole templating process was sort of “black magic” and a source of endless frustration (“%2F” problem)



Template Service (2)

- Has been completely rewritten
- Pluggable mappers allow reconfiguration of the mapping process
- Clear definition of rules how to write a template name and how it is mapped onto an actual template file and engine
- All template lookups are now made through the Template Service



Template Rules

- The golden rules of template names
  - "Template Names never contain slashes"
  - "Template names are no paths. They're not absolute and have no leading slash"
- These rules were present in prior Turbine versions but not enforced. This changed.



URI management in Turbine

- T2.2 contained a few classes to provide java representations of URIs
- These classes were disjunctive, duplicated code (and were full of bugs)
- Some of the classes doubled as Pull Tools for Template usage

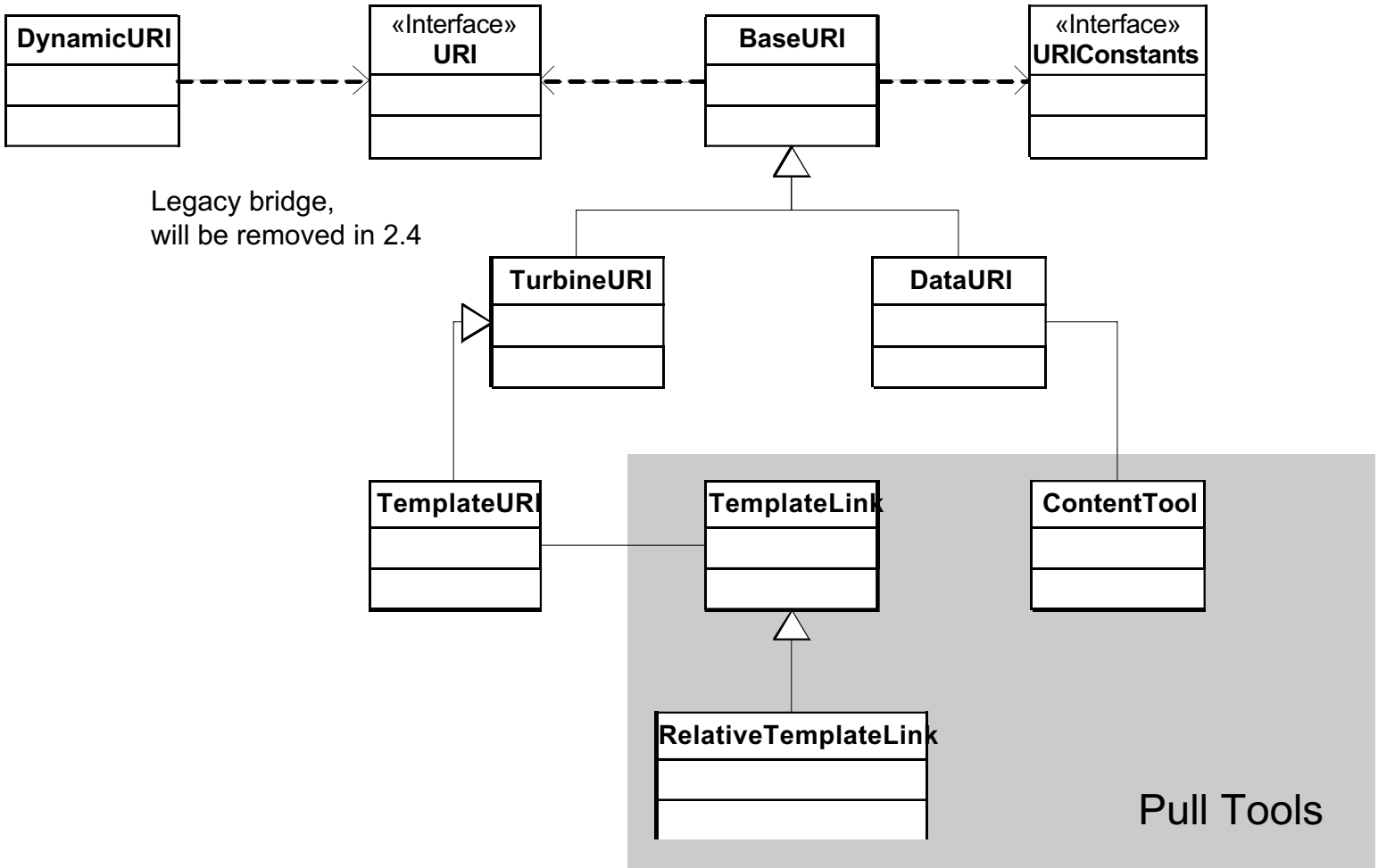


URI management in Turbine (2)

- T2.3 has now a hierarchical set of classes for URI representation and rules how to use them
- The org.apache.turbine.util.uri package contains eight classes for Java usage
- The Pull Service provides three tools for Template usage
- not yet using java.net.URI (JDK1.4)



URI management scheme



Security Service

- T2.2 Security Service can authenticate against databases and limited LDAP directories
- Rigid table scheme for database, no Torque based peers
- Extending the scheme is very difficult (almost no one got it right)



Security Service (2)

- 2.3 has a Torque based security service, using classes generated from XML
- All Security objects and ACL class are pluggable with interfaces and factories
- Uses Crypto Service
- Provide Security Objects with an “Id”

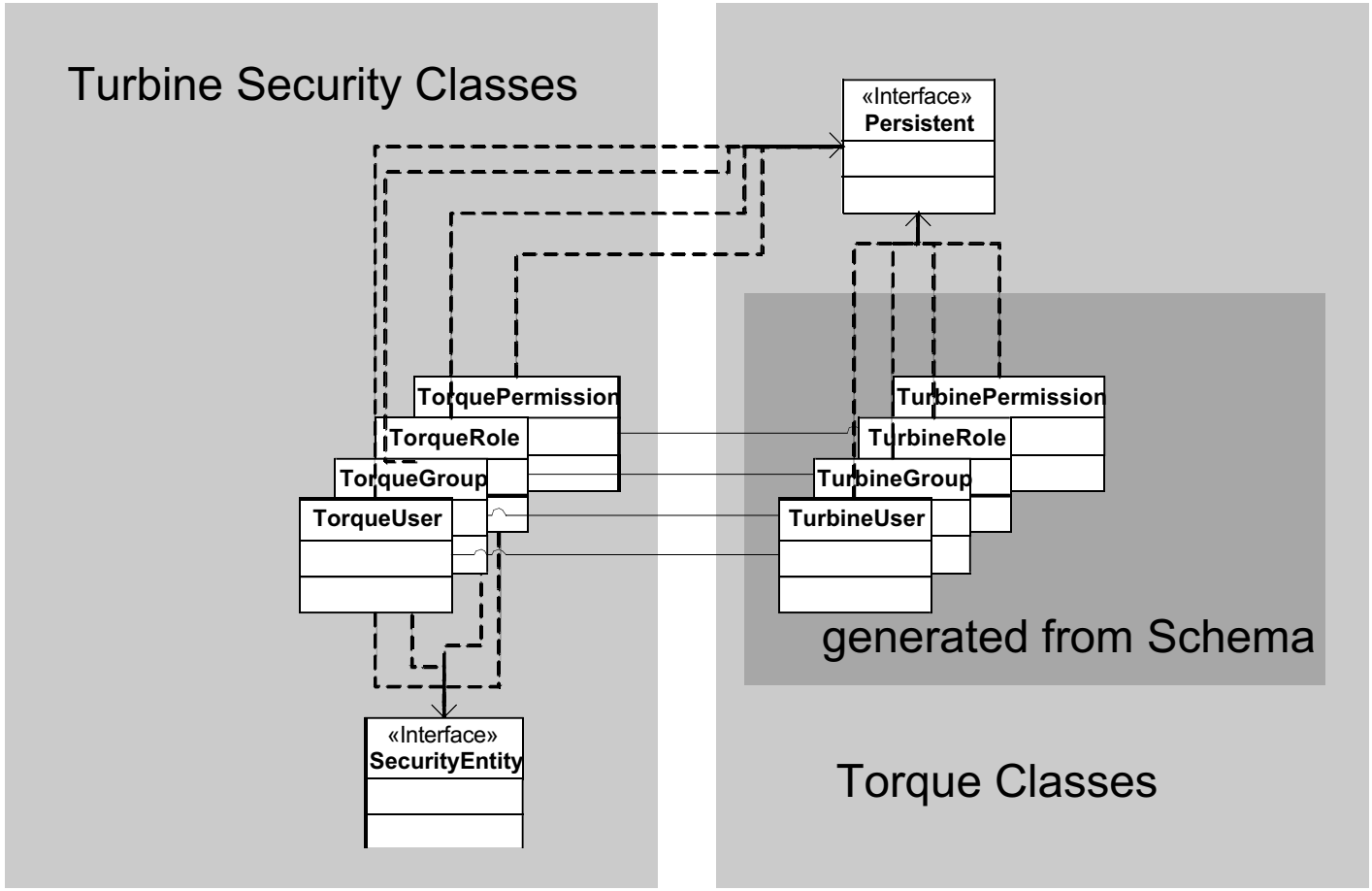


Torque Security Service

- Uses Torque peers to access SQL data
- No fixed scheme like the DB security service
- Splits data objects from security objects
- Makes extension or adaptation of the table scheme very easy



Torque Security Scheme





Extending a Security Object

Most of the time, the default columns are not sufficient

Add a new column to the existing Torque database scheme

```
<table name="TURBINE_USER" idMethod="idbroker">
[...]
  <column name="PHONE" type="VARCHAR" size="32"/>
[...]
</table>
```

Regenerate the Torque classes by running the torque:om maven task



Extending a Security Object (2)

Extending the Torque User class makes the column accessible from the application

```
package com.mycompany.security;

class public CustomUser extends TorqueUser {
  public String getPhone() throws TorqueException {
    return ((TurbineUser) getPersistentObj()).getPhone();
  }

  public void setPhone(String phone) throws TorqueException {
    ((TurbineUser) getPersistentObj()).setPhone(phone);
  }
}
```




Extending a Security Object (3)

Configure Turbine to use our new security object

TurbineResources.properties

```
[...]
services.SecurityService.user.class = \
                        com.mycompany.security.CustomUser
[...]
```




Summary

- Turbine 2.3 is the newest (and best!) release of the Turbine web framework
- While it contains numerous changes, all of them improve and facilitate the usage of the framework
- Improved Logging, Configuration and Security make development of Turbine based applications easier



Summary

- The addition of new Services like Crypto and AvalonComponent expand the “web tool box” that Turbine tries to be
- The rework of TemplateService and the URI classes should reduce developer frustration while developing and running Turbine applications

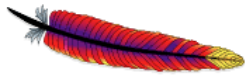


Any Questions?



Thanks a lot for your patience!

(even though the presentation was monochrome)



This presenation has ended.
Click anywhere to return.

_

